

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1442
Higher

COURSE NAME: Compiler Design for
Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom-up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Annexure A

Total Marks:50

DESIGN TASK

Code of Conduct:

I _K.Sabitha (192524308 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.Sabitha

Annexure A

DESIGN TASK

Q1. Predictive Parser Design (Application Level)

Task:

Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \varepsilon$

$F \rightarrow (E) \mid id$

Conditions:

- Remove left recursion (if any)
- Compute FIRST and FOLLOW sets
- Construct LL(1) parsing table

Test Case:

Input String:

id + id * id

Expected Output:

- Step-by-step parsing actions
- Final result: **String Accepted**

Code:

```
#include <stdio.h>
#include <string.h>
```

```
char input[50];
```

```
int pos = 0;
```

```
int match(char *str)
```

```
{
```

```
    int len = strlen(str);
```

```
    if (strncmp(&input[pos], str, len) == 0)
```

```
    {
```

```
        pos += len;
```

```
        return 1;
```

```
    }
```

```
    return 0;
```

```
}
```

```
int F()
```

```
{
```

```
    if (match("id"))
```

```
    {
```

```
        printf("F -> id\n");
```

```

        return 1;
    }
    return 0;
}

int Tdash()
{
    if (match("*"))
    {
        printf("T' -> *FT'\n");

        if (F())
            return Tdash();
        else
            return 0;
    }

    printf("T' -> ε'\n");
    return 1;
}

int T()
{
    printf("T -> FT'\n");

    if (F())
        return Tdash();

    return 0;
}

int Edash()
{
    if (match("+"))
    {
        printf("E' -> +TE'\n");

        if (T())
            return Edash();
        else
            return 0;
    }

    printf("E' -> ε'\n");
    return 1;
}

int E()
{
    printf("E -> TE'\n");

    if (T())
        return Edash();

    return 0;
}

```

```

}

int main()
{
    printf("Enter input (Example: id+id*id): ");
    scanf("%s", input);

    if (E() && input[pos] == '\0')
        printf("\nString Accepted\n");
    else
        printf("\nString Rejected\n");

    return 0;
}

```

Output:

```

Enter input (Example: id+id*id): id+id*id
E -> TE'
T -> FT'
F -> id
T' -> ε
E' -> +TE'
T -> FT'
F -> id
T' -> *FT'
F -> id
T' -> ε
E' -> ε

```

String Accepted

Q2. Shift-Reduce Parsing (Application Level)

Task:

Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:

$E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow id$

Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:

Input:

id + id * id

Expected Output:

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Code:

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[] = "id+id*id$";
    char stack[50] = "";
    int i = 0, top = 0;

    printf("Stack\t\tInput\t\tAction\n");

    while (input[i] != '$') {

        // Shift one character
        stack[top++] = input[i];
        stack[top] = '\0';

        printf("%-10s\t%-10s\tShift\n", stack, &input[i + 1]);

        // Reduce id -> E
        while (top >= 2 &&
               stack[top - 2] == 'i' &&
               stack[top - 1] == 'd') {

            top -= 2;
            stack[top++] = 'E';
            stack[top] = '\0';

            printf("%-10s\t%-10s\tReduce: id->E\n", stack, &input[i + 1]);
        }

        // Reduce E+E -> E
        while (top >= 3 &&
               stack[top - 3] == 'E' &&
               stack[top - 2] == '+' &&
               stack[top - 1] == 'E') {

            top -= 3;
            stack[top++] = 'E';
            stack[top] = '\0';

            printf("%-10s\t%-10s\tReduce: E+E->E\n", stack, &input[i + 1]);
        }

        // Reduce E*E -> E
        while (top >= 3 &&
               stack[top - 3] == 'E' &&
               stack[top - 2] == '*' &&
               stack[top - 1] == 'E') {
```

```

    top -= 3;
    stack[top++] = 'E';
    stack[top] = '\0';

    printf("%-10s\t%-10s\tReduce: E*E->E\n", stack, &input[i + 1]);
}

i++;
}

if (strcmp(stack, "E") == 0)
    printf("\nString Accepted\n");
else
    printf("\nString Rejected\n");

return 0;
}

```

Output:

Stack	Input	Action
i	d+id*id\$	Shift
id	+id*id\$	Shift
E	+id*id\$	Reduce: id->E
E+	id*id\$	Shift
E+i	d*id\$	Shift
E+id	*id\$	Shift
E+E	*id\$	Reduce: id->E
E	*id\$	Reduce: E+E->E
E*	id\$	Shift
E*i	d\$	Shift
E*id	\$	Shift
E*E	\$	Reduce: id->E
E	\$	Reduce: E*E->E

String Accepted

Q7. Operator Precedence Parsing Design and Evaluation

Task:

Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Conditions:

- Define operator precedence relations ($<$, $=$, $>$)
- Construct precedence table.
- Handle associativity rules explicitly.
- Detect and resolve conflicts.
- Show step-by-step parsing actions.
- Evaluate correctness of parsing decisions

Test Case:

Input: $\text{id} + \text{id} * \text{id}$.

Code:

```
#include<stdio.h>
#include<string.h>

int precedence(char op)
{
    if(op=='*')
        return 2;
    if(op=='+')
        return 1;
    return 0;
}

int main()
{
    char exp[50];
    char stack[50];
    int top=-1;

    printf("Enter Expression: ");
    scanf("%s",exp);

    printf("\nParsing Steps\n\n");

    for(int i=0;i<strlen(exp);i++)
    {
        if(exp[i]=='i' && exp[i+1]=='d')
        {
            printf("Shift id\n");
            i++;
        }
        else
        {
            while(top!=-1 && precedence(stack[top])>=precedence(exp[i]))
            {
                printf("Reduce using %c\n",stack[top]);
                top--;
            }

            stack[++top]=exp[i];

            printf("Shift %c\n",exp[i]);
        }
    }
}
```

```

    }
}

while(top!=-1)
{
    printf("Reduce using %c\n",stack[top]);
    top--;
}

printf("\nString Accepted");

return 0;
}

```

Output:

Enter Expression: id+id*id

Parsing Steps

```

Shift id
Shift +
Shift id
Shift *
Shift id
Reduce using *
Reduce using +

```

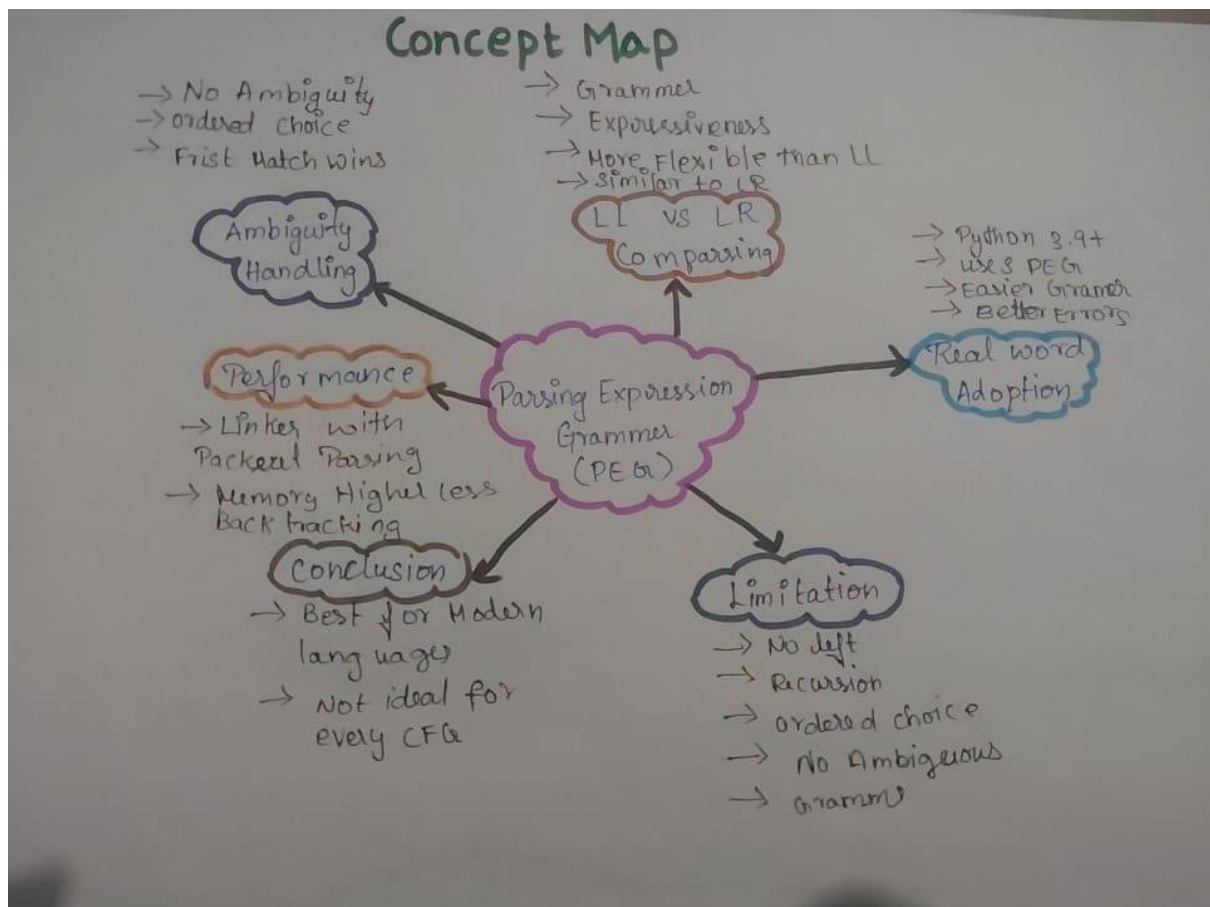
String Accepted

Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers
2. **Compare** PEG with LL and LR parsing in terms of:
 - Grammar expressiveness
 - Ease of implementation
 - Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification

Concept map:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation